

[Ville Voutilainen](#)

[Jonathan Wakely](#)

[John Spicer](#)

[Stephan T. Lavavej](#)

2025-10-07

C++26 Contracts are not a good fit for standard library hardening

Abstract

The C++26 IS draft specifies that hardened preconditions may be, in a hardened implementation, evaluated with a non-terminating semantic. This can result in violations of hardened preconditions being undefined behaviour, rather than guaranteed to be diagnosed, which defeats the purpose of using a hardened implementation.

Hand-waving about implementation-defined means and mechanisms for selecting contract-evaluation semantics for individual contracts doesn't help; some standard library implementations need to work with multiple compilers, and there's no known way to request multiple different compilers to make sure that the hardened preconditions are never evaluated with a checked but non-terminating semantic (i.e. 'observe').

The outcome of this is that despite best intentions, C++26 standard library implementations cannot implement hardened preconditions in terms of contracts. And there's another reason why they can't - standard library hardening must be allowed to be turned on regardless of what the evaluation semantics of contracts in the rest of the program and even in the same TU are.

Integrating C++26 contracts into standard library hardening is an unimplemented specification exercise that must be undone before C++26 ships.

This concern is nowhere to be seen in P3846

There are two NB comments asking for contracts to be decoupled from standard library hardening, and those are FR-001-014 ([github](#)) and US 3-015 ([github](#)).

[P3846](#) doesn't even mention these NB comments, and doesn't address them.

A library-wide 'observe' mode is not a thing for a hardened implementation

The specification says, in [structure.specifications]/3.5.1, with the problematic part bolded,

When invoking the function in a hardened implementation, prior to any other observable side effects of the function, one or more contract assertions whose predicates are as described in the hardened precondition are evaluated with a checking semantic (6.11.2). **If any of these assertions is evaluated with a non-terminating semantic and the contract-violation handler returns, the program has undefined behavior.**

That is completely unacceptable for a hardened implementation. A hardened implementation is a memory safety mechanism, and it's not memory-safe to run into the UB that results in calling a library function violating its preconditions.

Yes, there are ostensible use cases for having observed hardened preconditions, for code that's common in the wild, doesn't actually run into abstract machine UB, and works fine. The current specifications of hardened preconditions leave those cases out due to the language not providing a way to select specific semantics for individual contracts, portably. Such cases are an exception to the general rule. As an example of such a use case, consider `a[n]` (or rather, `&vec[vec.size()]`), for a very concrete example that apparently appears in the wild) for a sequence container. It has a hardened precondition "`n < a.size()` is true". We could give it an always-terminating precondition "`n <= a.size()`", and an observed precondition "`n < a.size()`" if we could express that with the language facility, but we can't do that yet.

But that's a far cry from somehow being able to select an observe semantic for other hardened preconditions. They have one purpose, and only one, and that is to **guarantee** that violating them doesn't lead to UB. Ever. That's what hardened implementations do with them,

and that's what hardened implementations will need to continue to do with them, without compromise of any kind, be it whatever other choice for contract semantics is made for other code or something else, even in the same TU.

The wished-for "grand unification" doesn't work, and cannot work

P3846 suggests that there should be "Consistency across libraries". What it seems to mean by that is that all checking mechanisms are unified under the same control mechanism, and in the C++26 timeframe, that means whatever mechanism for selecting contract evaluation semantics.

That doesn't work. It's a requirement of paramount and non-negotiable importance to be able to select a hardened standard library implementation regardless of what's done with any and all other checking mechanisms. And when such a hardened implementation is selected, it must never continue into the stdlib code in case of hardened precondition violations, it must make sure that that code isn't entered. It needs to do that now, forever, and without any "maybe" period in between.

That means that the hardened preconditions are, and have to be, forever decoupled from the rest of the program, including the use of whatever checking mechanisms (contracts on user code, various forms of asserts in/on user code) in the same TU. You can have your own contracts, and set them to ignore or observe, or enforce or quick_enforce, and you can still rely on the hardened preconditions not to continue into the stdlib code and cause UB. Not all of that UB is just benign dereferences of null pointers that the OS will catch for you anyway. That is, if you're even using an OS.

Is this QoI?

A stdlib implementation can certainly make sure that the hardened preconditions are always evaluated with a terminating semantic. But that means that semantic is quick_enforce. C++26 contracts do not have a facility that allows calling a possibly replaced violation handler (they do allow invoking the implementation-defined default handler, but not a replaced one), so what an implementation would need to do is not use contracts the language facility in its code, and just terminate directly, without invoking any violation handler.

But the problem of 'observe' being a possibility allowed by the specification remains. Leaving that possibility of observe in is just silly. No standard library implementation should allow for that possibility, it should just be removed from the specification.

A C++ implementation would have two options, if it tried to use contracts:

1. Specify library-precondition-specific evaluation semantics for the hardened preconditions. This option isn't viable, because a mechanism for doing that doesn't exist in C++26 contracts, and it's also infeasible to try to rely on compiler-specific mechanisms for portable stdlib implementations that need to work with multiple compilers, and in multiple modes. Libstdc++, libc++, and MSVC STL support at least three compilers each, so cannot make use of compiler-specific mechanisms for using a particular semantic for hardened preconditions even when the rest of the program uses 'ignore'.
2. Make it so, with the help of the compiler, that when hardening is enabled, all contracts in the TU are given a terminating evaluation semantic. This is
 - draconian, it's unreasonable to do that also for all user contracts in the user's own code
 - infeasible still, because standard library hardening can be enabled separately from the rest of the hardening facilities that compilers provide, so relying on the shortcut selection for a bunch of hardening options (i.e. 'g++ -fhardened') still doesn't do the right thing, and doesn't suffice.

A far more straightforward option is to just not use contracts.

The stdlib hardening paper changed between R2 and the approved R4. Originally it was unspecified how the checks were done, only that a failure would invoke a handler and then terminate. That had the correct desired semantics.

Rumination summary

A hardened C++26 standard library implementation cannot be implemented using Contracts the language facility. The language facility doesn't provide what a hardened stdlib implementation needs.

Wording

Drafting note: this wording strives for a minimal change to the wording. It doesn't attempt to provide a resolution to the NB comments that ask for standard library hardening to be completely decoupled from contracts. This wording does allow such a completely decoupled implementation, by implementing essentially a hard-coded quick_enforce.

In [structure.specifications]/3.1.5, edit as follows:

When invoking the function in a hardened implementation, prior to any other observable side effects of the function, ~~one or more~~ contract assertions whose predicates are as described in the hardened precondition are evaluated with a ~~checking~~terminating semantic (6.11.2). ~~If any of these assertions is evaluated with a non-terminating semantic and the contract-violation handler returns, the program has undefined behavior.~~